



THE OS FOR MULTI-AGENT DEVELOPMENT

synlynk

quick start.

From zero to a context-aware, cost-tracked,
multi-agent workgroup in under two minutes.

v0.4.1 Instruction Reach

```
Terminal

$ synlynk init

✓ Project context injected
✓ 3 agents discovered · claude · agy · codex
✓ Budget guard active · $10.00 / session
✓ Sentinel monitoring enabled

→ synlynk exec claude
```

Zero dependencies

Python 3 stdlib only

Claude · AGY · Codex

Instruction reach

Cost tracking

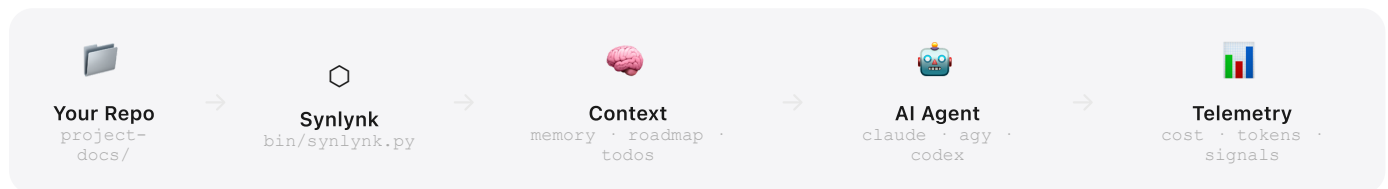
Sentinel guard

Background dispatch

WHAT IS SYNLYNK?

Your AI agents, working together.

Synlynk is a single-file Python CLI that wraps AI agents — Claude, AGY, Codex — and makes them project-aware, cost-tracked, and safe to run in parallel. One file. Zero dependencies. Works anywhere Python 3 runs.



Why Synlynk?



Context-Aware by Default

Every agent invocation automatically receives your roadmap, memory, and todos. No more copy-pasting context into every session.



Zero Surprise Bills

Budget guard blocks execution the moment you hit your daily or session limit. Real-time cost tracking on every exec.



Hallucination Sentinel

Automatically detects flatline loops, success loops, and quota exhaustion. Alerts before damage is done.



Parallel Dispatch

Launch Claude, Gemini, and Codex simultaneously on different tasks. Monitor all jobs from one CLI while your terminal stays free.



Team-Ready

Single or team mode. Per-user devlogs with git attribution. Shared roadmap and memory visible to every agent in the repo.



Zero Dependencies

Pure Python 3 stdlib. No pip install, no Docker, no build step. One file — works in any environment where Python 3 runs.

1

FILE — NO BUILD STEP

3

AI AGENTS SUPPORTED

0

PIP DEPENDENCIES

∞

PARALLEL AGENT JOBS

GETTING STARTED

Up and running in 60 seconds.

1 Clone & install globally

Adds `synlynk` to your PATH via `~/.synlynk/bin/`. No sudo, no pip, no build step required.

```
install

$ git clone https://github.com/nikhilsoman/synlynk
$ cd synlynk && ./install.sh

✓ Installed to ~/.synlynk/bin/synlynk
✓ PATH updated in ~/.zshrc
✓ synlynk 0.4.1 ready
```

2 Bootstrap any project

Run inside any repo. Creates `project-docs/`, writes agent instruction files for all detected IDEs and CLIs, and generates a SHA manifest. Safe to re-run — appends to existing files without overwriting user content.

```
synlynk init

~/my-project $ synlynk init

Mode: single | team? single

✓ project-docs/roadmap.md
✓ project-docs/memory.md
✓ project-docs/todo.md & costs.md
✓ CLAUDE.md · GEMINI.md · AGENTS.md (trio instruction files)
✓ .cursor/rules/synlynk.mdc · .windsurfrules (IDE reach)
✓ Agents: claude · agy · codex

→ synlynk exec claude
```

3 Run your first context-injected session

Synlynk prepends your project context — the AI already knows your roadmap, decisions, and open tasks before you type a word.

```
synlynk exec

$ synlynk exec claude

📁 Context injected (memory · roadmap · todos)
💰 Budget: $2.34 / $10.00 used this session
🚀 Launching claude...

claude > I can see you're building a billing API for the
```

monetization module. Based on your roadmap...

- **Magic Moments**

The init wizard scans your README, git history, and existing docs to pre-fill your roadmap and memory with real context — not generic placeholders.

V0.4.0 · HYBRID WORKGROUP BOOTSTRAP

Run agents in parallel.

In the background.

Dispatch fires an agent as a background job — captured to a log file, tracked by the job store. Multiple agents run simultaneously while your terminal stays free.

```
synlynk dispatch

$ synlynk dispatch claude --task "Write unit tests for the billing module"
✓ Dispatched job-a3f2 (claude) running in background

$ synlynk dispatch agy --task "Review billing API for edge cases"
✓ Dispatched job-b7e1 (agy) running in background

# Both agents working simultaneously — terminal is yours
```

Monitor all jobs

```
synlynk jobs

$ synlynk jobs

• job-a3f2 claude running 2m14s
• job-b7e1 agy running 1m58s
```

Tail a job's output

```
synlynk logs

$ synlynk logs job-a3f2

Writing test_billing.py...
✓ test_charge_success
✓ test_refund_partial
→ 12 / 12 tests passing
```

The Trio Pipeline v0.4.0

Three agents in sequence — Architect designs, Builder implements, Verifier reviews — with full context hand-off between each phase.

```
synlynk run --trio

$ synlynk run --trio "Add Stripe webhook handler"

Phase 1 Architect (claude) ... ✓ done job-c1a0
Phase 2 Builder (agy) ... ✓ done job-d4b2
Phase 3 Verifier (claude) ... ✓ done job-e9f3

✓ Trio complete. synlynk logs job-e9f3
```

Live in v0.5.0 — Smart Routing

Dispatch automatically routes to the best-performing agent for each task domain, based on historical capability scores from a SQLite ledger. Use `synlynk story create` and `synlynk score add` to build up routing data.

SENTINEL & BUDGET GUARD

Your safety net, always on.

Synlynk watches every agent invocation for danger signals and surfaces alerts before they become expensive mistakes.

Budget Guard

Set spending limits in `.synlynk/config.json`. Synlynk blocks the next exec the moment you'd exceed your limit. Cost shown after every run.

```
.synlynk/config.json
{
  "limit_usd": 10.00,
  "limit_requests": 100
}

budget pulse

— Budget Pulse —
Cost:      $2.34 / $10.00
Requests: 14 / 100
Tokens:    2,840 in · 480 out
```

Sentinel Patterns

Sentinel watches your telemetry log and fires alerts automatically for four danger patterns.

- FLATLINE** **Same command failed 3x in a row**
Possible hallucination loop. Manual intervention recommended before more tokens burn.
- LOOP** **Same command succeeded 5x in <10 min**
Likely an automated loop running unproductively. Investigate before continuing.
- QUOTA** **Quota exhaustion detected in output**
Output matched a known quota-exhausted phrase. Switch agent or check plan limits.
- ZOMBIE** **Job PID dead but never marked complete**
Or job has been running >30 min without output (STALL). Reconciled automatically on next run.
- DRIFT** **Instruction file edited outside synlynk**
A tracked file's synlynk section was modified externally. Run `synlynk instructions update` or `ack` to resolve.

Managing Alerts

```
sentinel commands

$ synlynk sentinel list

[CRITICAL] FLATLINE      `synlynk exec claude` failed 3x — 2026-06-14 09:41
[WARN]    SUCCESS_LOOP  Same command 5x in 4.2 min — 2026-06-14 10:02

$ synlynk sentinel clear --code FLATLINE
✓ Cleared FLATLINE alerts
```

ALL COMMANDS

Everything at a glance.

Setup & Context

COMMAND	WHAT IT DOES
<code>synlynk init</code>	Bootstrap <code>project-docs/</code> , agent instruction files, and <code>.synlynk/config.json</code> in any repo. Safe to re-run — skips existing files.
<code>synlynk exec <agent></code>	Run an agent interactively with full context injection (memory + roadmap + todos prepended). Captures token counts and cost on exit.
<code>synlynk checkpoint</code>	Force a context snapshot — regenerates <code>.synlynk/context.md</code> from current project-docs state.
<code>synlynk status</code>	Dashboard showing current project state, recent costs, active sentinel alerts, and running jobs.
<code>synlynk upgrade</code>	Check for and apply updates from the remote repository.

Dispatch & Jobs v0.4.0

COMMAND	WHAT IT DOES
<code>synlynk dispatch <agent></code>	Fire an agent task in the background. Captures stdout to a log file. Returns a job ID immediately. Accepts <code>--task "..."</code> and <code>--story <id></code> .
<code>synlynk jobs</code>	List all tracked jobs with status (running / completed / failed), agent, and duration. Use <code>--all</code> to include completed jobs.
<code>synlynk logs <job-id></code>	Tail the output log of a specific job. Use <code>--tail N</code> to control how many lines to show.
<code>synlynk run --trio</code>	Run a three-phase Architect → Builder → Verifier pipeline with context hand-off between each phase.

Sentinel

COMMAND	WHAT IT DOES
<code>synlynk sentinel list</code>	Show all active alerts with severity, code, message, and timestamp.
<code>synlynk sentinel clear</code>	Clear alerts. Target with <code>--severity CRITICAL</code> or <code>--code FLATLINE</code> .

Instructions v0.4.1

COMMAND	WHAT IT DOES
<code>synlynk instructions status</code>	Show status of each tracked instruction file: ok / missing / drifted / untracked / no-markers.
<code>synlynk instructions diff</code>	Show user content outside synlynk markers in each tracked file.
<code>synlynk instructions update <file></code>	Regenerate the synlynk section in a file and reset its SHA in the manifest.
<code>synlynk instructions ack <file></code>	Remove INSTRUCTION_DRIFT lines from sentinel.md without regenerating.

project-docs/

```
├─ roadmap.md    ← version plan
├─ memory.md     ← decisions
├─ todo.md       ← open tasks
├─ costs.md      ← cost ledger
└─ devlogs/
    └─ nikhil.md ← per-user log
```

.synlynk/

```
├─ config.json    ← budget limits
├─ context.md     ← auto-generated
├─ telemetry.json ← rolling log
├─ instructions.json ← SHA manifest
└─ logs/          ← job outputs
```

Every `synlynk exec` prepends `memory.md` + `roadmap.md` + `todo.md` as a single context snapshot before handing off to the agent.

- **Keep project-docs current**

Synlynk's context is only as good as your `memory.md` and `roadmap.md`. Update them after significant decisions — agents pick up changes on the next exec automatically.

- **Smart routing is live in v0.5.0**

`synlynk dispatch` and `synlynk run --trio` now route automatically based on capability scores. Use `synlynk story create` and `synlynk score add` to build up routing data across your team.

WHAT'S COMING NEXT

The road to v1.0.

Synlynk evolves from an agent wrapper into a full OS for multi-agent development. Each release adds a new layer of the stack.

- ▼0.5

Capability Engine

✓ Live · June 2026

Model-aware routing · 3D domain taxonomy · quality signal hierarchy · SQLite state ledger
- ▼0.6

Job Control + Constraints

✓ Live · June 2026

Constraint propagation · job state machine · PR attestation · verifier pipeline capture
- ▼0.7

Async Pipeline + Daemon

Oct 2026

Background daemon · HTTP context server · synlynk review TUI
- ▼0.8

Open Context Protocol

Nov 2026

context --for · checkpoint --from · MCP server · ecosystem interface
- ▼1.0

Stable OS + Tokq Bridge Ready

Q1 2027

Frozen CLI · pipx/Homebrew · NATS leaf node schema · capability marketplace bridge